Announcements

After this lecture you're ready to start project 0: Particle Simulator.

- Due next Tuesday.
- Recommendation: Start once you've finished this week's discussion worksheet.
  - Don't wait too much longer. The project is long if you're new to Java.

HW and survey weights tweaked:

- HW now 300 points (up from 280).
- Survey now down to 100 points (up from 20).
- This was to make the numbers work out nicer.
- Number of HWs up from 6 to 8.
  - Each is worth 50 points.
  - Can lose up to 100 points in homework category and still get a full score.

CC BY NC SA

Announcements

Discussion and labs are assigned.

- Post to Ed if you're waiting.
- Use Sections link (top right) to change your section or lab.



One time only: I'll teach a section today in Cory, 3 PM - 4 PM in Cory 293

- If you weren't matched with any section or you don't like your section time feel free to come today.

CC BY NC SA



Lecture 3

Lists, Arrays, Maps, and References

CS61B, Fall 2026 @ UC Berkeley

Josh Hug and Manuel Sabin

CC BY NC SA

Modern Java Lists

Lecture 3, CS61B, Fall 2026

Three Important Data Structures: Lists, Arrays, and Maps

- **Modern Java Lists**
- Arrays
- Maps

References

- Memory Box Model
- Reference Types
- Parameter Passing
- 2D Arrays

CC BY NC SA

Retrieving from Lists

Below, we see our (old school) Java List code from the previous lecture. It works, but we'll see a serious limitation if we try to store the results of a retrieval.

```
import java.util.ArrayList;
import java.util.List;

void main() {
    List L = new ArrayList();
    L.add("a");
    L.add("b");
}
```

CC BY NC SA

Retrieving from Lists

Retrieval from a list:

- In Python, we retrieve from a list using bracket notation.

```
L = []
L.append("a")
L.append("b")
print(L[0])
```

```
a
```

```
import java.util.ArrayList;
import java.util.List;

void main() {
    List L = new ArrayList();
    L.add("a");
    L.add("b");
}
```

CC BY NC SA

Retrieving from Lists

Retrieval from a list:

- In Python, we retrieve from a list using bracket notation.
- In Java, we retrieve using the `get` method.

```
L = []
L.append("a")
L.append("b")
print(L[0])
```

```
a
```

```
import java.util.ArrayList;
import java.util.List;

void main() {
    List L = new ArrayList();
    L.add("a");
    L.add("b");
    System.out.println(L.get(0));
}
```

CC BY NC SA

Java 4.0 List Limitation

The list we created is a Java 4.0 style list (from Java before 2005).

- Serious limitation: Can't (easily) assign the results of retrieving an item from a list to a variable.
- I won't teach you the Java 4.0 way to deal with this, because it is obsolete.

```
L = []
L.append("a")
L.append("b")
print(L[0])
```

Won't compile. Results in a "Required type: String, Provided: Object" error.

```
import java.util.ArrayList;
import java.util.List;

void main() {
    List L = new ArrayList();
    L.add("a");
    L.add("b");
    String x = L.get(0);
}
```

CC BY NC SA

Java 5.0 Lists

In 2005, a new angle bracket syntax was introduced to deal with this issue.

- Instead of List L, the programmer specifies the specific type of list with List<String> L.
- Requires <> on the instantiation side.

```
L = []
L.append("a")
L.append("b")
print(L[0])
```

```
import java.util.ArrayList;
import java.util.List;

void main() {
    List<String> L = new ArrayList<>();
    L.add("a");
    L.add("b");
    String x = L.get(0);
}
```

Historical note: You had to have <String> on both declaration and instantiation sides until 2011.

CC BY NC SA

Java 5.0 Lists

All items in a list created using angle bracket notation must have the same type!

- Unlike Python lists which can store multiple types.

```
L = []
L.append("a")
L.append(3)
# fine
```

```
import java.util.ArrayList;
import java.util.List;

void main() {
    List<String> L = new ArrayList<>();
    L.add("a");
    L.add(3);
}
```

Syntax error. Won't compile.



CC BY NC SA

Limitations

Going from Java 4.0 to Java 5.0 lists meant losing the ability to have lists with multiple types.

Why might this be a good thing?

-

CC BY NC SA

Limitations (My Answer)

Going from Java 4.0 to Java 5.0 lists meant losing the ability to have lists with multiple types.

Why might this be a good thing?

- It restricts the set of choices you have to make as a programmer.

Placing limitations on yourself as a programmer is a good thing!

- Freedom leads to complexity.
- Complexity is hard to fit in your brain.
- Java has MANY features that help you restrict yourself.
- Will be a recurring theme in this class.

CC BY NC SA

Arrays

Lecture 3, CS61B, Fall 2026

Three Important Data Structures: Lists, Arrays, and Maps

- Modern Java Lists
- **Arrays**
- Maps

References

- Memory Box Model
- Reference Types
- Parameter Passing
- 2D Arrays

CC BY NC SA

Arrays

Java has a special collection called an “array” that is a restricted version of the list ADT.

- Size that must be declared at the time the array is created.
- Size cannot change.
- All items must be of the same type.
- No methods.
- Syntax for accessing array entries is similar to Python, e.g. x[0].

No Python equivalent.

- (that you’ve learned)

You can call x.length.

```
void main() {
    String[] x = new String[5]; // size 5
    x[0] = "a";
    x[1] = "b";
    System.out.println(x[0]);
}
```

CC BY NC SA

Arrays

Like classes, arrays are (almost always) instantiated with new.

Three valid notations:

```
x = new int[3];
y = new int[]{1, 2, 3, 4, 5};
int[] z = {9, 10, 11, 12, 13};
```

Creates array of length 3 (with all zeros by default).

Can omit the new if you are also declaring a variable.

Note: Arrays have no methods and exactly one instance variable called `length`.

- To check the length of an array, use `x.length`.

CC BY NC SA

Question for You

Why do you think Java has both lists and arrays (basically lists with fewer features)?

CC BY NC SA

Question for You (My Answers)

Why do you think Java has both lists and arrays?

- Arrays are more performant:
  - Reading and writing from them is faster.
  - Use less memory.
- You'll learn more about this in 61C.

CC BY NC SA

Question for You

Why do you think Java favors arrays over lists (has special bracket syntax, doesn't require imports)?

CC BY NC SA

Question for You (My Answer)

Why do you think Java favors arrays over lists (has special bracket syntax, doesn't require imports)?

- Java is a language built for performance.
- By contrast, Python is built to be beautiful/simple/elegant.

Side note, when you take 61C, you'll learn C.

- C is more challenging to write than Java.
- C is more oriented around performance.

CC BY NC SA

Maps

Lecture 3, CS61B, Fall 2026

Three Important Data Structures: Lists, Arrays, and Maps

- Modern Java Lists
- Arrays
- **Maps**

References

- Memory Box Model
- Reference Types
- Parameter Passing
- 2D Arrays

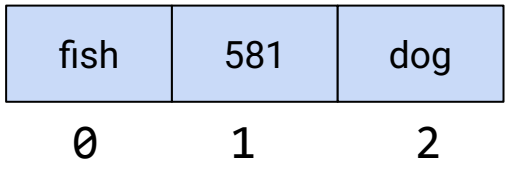
CC BY NC SA

Maps in Python

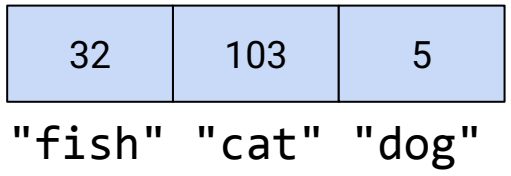
Below, we see examples of a list and a dictionary (a.k.a. map) in Python.

- We can think of a map as a generalization of a list.
  - Lists map integers to data, e.g. L[0] gets the 0th item of the list called L.
  - Maps map ANY KEY type to data, e.g. census[“cat”] gets the catth of the map called census.

```
L = ["fish"]
L.append(581)
L.append("dog")
item0 = L[0] # fish
```




```
census = {"fish" : 32}
census["cat"] = 103
census["dog"] = 5
num_cats = census["cat"]
```



CC BY NC SA

Maps

In programming languages, a map is a collection of key-value pairs. Each key is guaranteed to be unique. Also called:

- Dictionary (in Python).
- Associative Array (in theoretical computer science).

Example:
```
{"alpha": "first letter of greek alphabet",
 "potato": "a starchy edible plant"}
```

For more, see: https://en.wikipedia.org/wiki/Associative_array

CC BY NC SA

Maps in Python

Let’s see how we’d translate the code below into Java.

```
census = {}
census["cat"] = 103
census["dog"] = 5
num_cats = census["cat"]
```

CC BY NC SA

Practice with Lists, Arrays, Maps

In this week's discussion you'll get practice with lists, maps, and arrays.

- You're welcome to work through the worksheet even if you're not on the discussion track.
- Solutions Wednesday.

```
census = {}
census["cat"] = 103
census["dog"] = 5
num_cats = census["cat"]
```

```
import java.util.HashMap;
import java.util.Map;

void main() {
    Map<String, Integer> m = new HashMap<>();
    m.put("cat", 103);
    m.put("dog", 5);
    int numCats = m.get("cat");
}
```

CC BY NC SA

Memory Box Model

Lecture 3, CS61B, Fall 2026

Three Important Data Structures: Lists, Arrays, and Maps

- Modern Java Lists
- Arrays
- Maps

References

- **Memory Box Model**
- Reference Types
- Parameter Passing
- 2D Arrays

CC BY NC SA

Random Question (which I haven't taught you how to solve)

```
Walrus a = new Walrus(1000, 8.3);
Walrus b;
b = a;
b.weight = 5;
System.out.println(a);
System.out.println(b);
```

Will the change to b affect a?

A. Yes
B. No



hugcode.com/hats

CC BY NC SA

Random Question (which I haven't taught you how to solve)



hugcode.com/lemon

```
int x = 5;
int y;
y = x;
x = 2;
System.out.println("x is: " + x);
System.out.println("y is: " + y);
```

Will the change to x affect y?

A. Yes

B. No

CC BY NC SA

Random Question (which I haven't taught you how to solve)

```
Walrus a = new Walrus(1000, 8.3);
Walrus b;
b = a;
b.weight = 5;
System.out.println(a);
System.out.println(b);
```

Will the change to b affect a?

A. Yes
B. No

```
weight: 5, tusk size: 8.30
weight: 5, tusk size: 8.30
```

```
int x = 5;
int y;
y = x;
x = 2;
System.out.println("x is: " + x);
System.out.println("y is: " + y);
```

Will the change to x affect y?

A. Yes
B. No

```
x is: 2
y is: 5
```

Answer: Visualizer

CC BY NC SA

Bits

Your computer stores information in “memory”.

- Information is stored in memory as a sequence of ones and zeros.
  - Example: 72 stored as 01001000
  - Example: 205.75 stored as … 01000011 01001101 11000000 00000000
  - Example: The letter H stored as 01001000 (same as the number 72)
  - Example: True stored as 00000001

Each Java type has a different way to interpret the bits:

- 8 primitive types in Java: byte, short, **int**, long, float, **double**, boolean, char
- We won’t discuss the precise representations in much detail in 61B.
  - Covered in much more detail in 61C.

Note: Precise representations may vary from machine to machine.

CC BY NC SA

Declaring a Variable (Simplified)

When you declare a variable of a certain type in Java:

- Your computer sets aside exactly enough bits to hold a thing of that type.
  - Example: Declaring an int sets aside a “box” of 32 bits.
  - Example: Declaring a double sets aside a box of 64 bits.
- Java creates an internal table that maps each variable name to a location.
- Java does NOT write anything into the reserved boxes.
  - For safety, Java will not let you access a variable that is uninitialized.

```
int x;
double y;
x = -1431195969;
y = 567213.112;
```



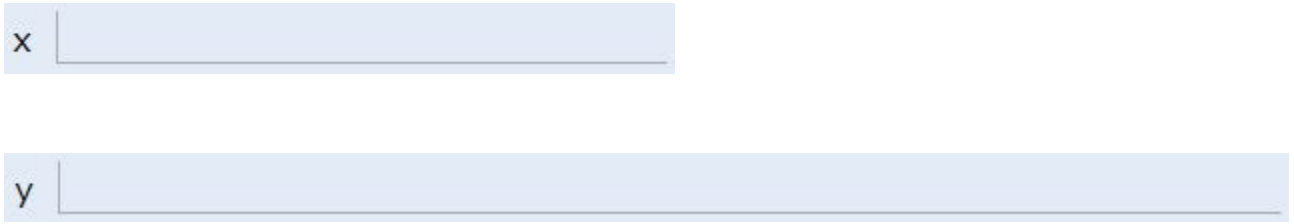
CC BY NC SA

Declaring a Variable (Simplified)

When you declare a variable of a certain type in Java:

- Your computer sets aside exactly enough bits to hold a thing of that type.
  - Example: Declaring an int sets aside a “box” of 32 bits.
  - Example: Declaring a double sets aside a box of 64 bits.
- Java creates an internal table that maps each variable name to a location.
- Java does NOT write anything into the reserved boxes.
  - For safety, Java will not let you access a variable that is uninitialized.

```
int x;
double y;
x = -1431195969;
y = 567213.112;
```



CC BY NC SA

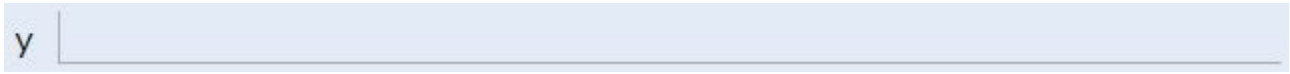
Declaring a Variable (Simplified)

When you declare a variable of a certain type in Java:

- Your computer sets aside exactly enough bits to hold a thing of that type.
  - Example: Declaring an int sets aside a “box” of 32 bits.
  - Example: Declaring a double sets aside a box of 64 bits.
- Java creates an internal table that maps each variable name to a location.
- Java does NOT write anything into the reserved boxes.
  - For safety, Java will not let you access a variable that is uninitialized.

```
int x;
double y;
x = -1431195969;
y = 567213.112;
```





CC BY NC SA

Declaring a Variable (Simplified)

When you declare a variable of a certain type in Java:

- Your computer sets aside exactly enough bits to hold a thing of that type.
  - Example: Declaring an int sets aside a “box” of 32 bits.
  - Example: Declaring a double sets aside a box of 64 bits.
- Java creates an internal table that maps each variable name to a location.
- Java does NOT write anything into the reserved boxes.
  - For safety, Java will not let you access a variable that is uninitialized.

```
int x;
double y;
x = -1431195969;
y = 567213.112;
```





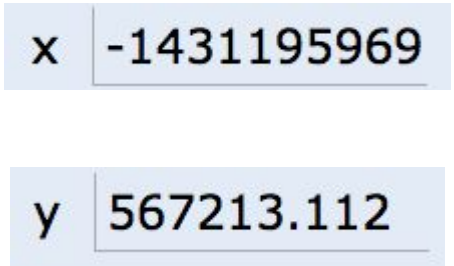
CC BY NC SA

Simplified Box Notation

We'll use simplified box notation from here on out:

- Instead of writing memory box contents in binary, we'll write them in human readable symbols.

```
int x;
double y;
x = -1431195969;
y = 567213.112;
```



CC BY NC SA

The Golden Rule of Equals (GRoE)

Given variables y and x:

- y = x **copies** all the bits from x into y.

Example from earlier: Link

CC BY NC SA

Reference Types

Lecture 3, CS61B, Fall 2026

Three Important Data Structures: Lists, Arrays, and Maps

- Modern Java Lists
- Arrays
- Maps

References

- Memory Box Model
- **Reference Types**
- Parameter Passing
- 2D Arrays

CC BY NC SA

Reference Types

There are 8 primitive types in Java:

- byte, short, **int**, long, float, **double**, boolean, char

Everything else, including arrays, is a **reference type**.

CC BY NC SA

Class Instantiations

When we instantiate an Object (e.g. Dog, Walrus, Planet):

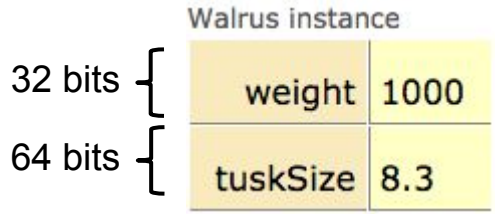
- Java first allocates a box of bits for each instance variable of the class and fills them with a default value (e.g. 0, null).
- The constructor then usually fills every such box with some other value.

```
public static class Walrus {
   public int weight;
   public double tuskSize;

   public Walrus(int w, double ts) {
      weight = w;
      tuskSize = ts;
   }
}
```

```
new Walrus(1000, 8.3);
```

Demo Link



CC BY NC SA

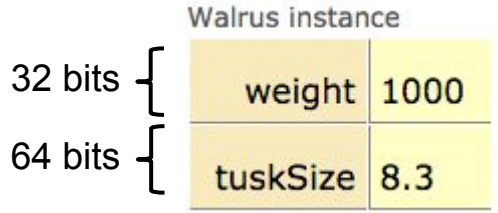
Class Instantiations

When we instantiate an Object (e.g. Dog, Walrus, Planet):

- Java first allocates a box of bits for each instance variable of the class and fills them with a default value (e.g. 0, null).
- The constructor then usually fills every such box with some other value.

```
....00000000000000000000000000000000000000
00000000000000000000000000000000000000000
00000000000000000000000000000000000000000
00000000000000000000000000000000001111101
00001000000001000001001100110011001100110
01100110011001100110100000000000000000000
00000000000000000000000000000000000000000
00000000000000000000000000000000000000000
00000000000000000000000000000000000000000
00000000000000000000000000000000000000000
00000000000000000000000000000000000000000
00000000000000000000000000000000000000000
00000000000000000000....
```

```
new Walrus(1000, 8.3);
```



Green is weight, blue is tuskSize.

(In reality, total Walrus size is slightly larger than 96 bits.)

CC BY NC SA

Class Instantiations

Can think of `new` as returning the address of the newly created object.

- Addresses in Java are 64 bits.
- Example (rough picture): If object is created in memory location 2384723423, then new returns 2384723423.

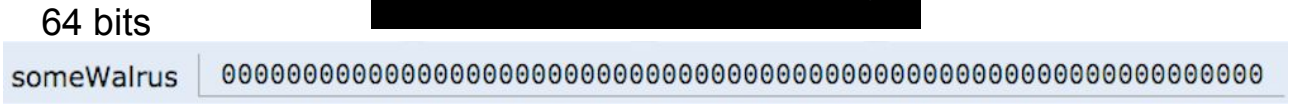

2384723423th bit
....00000000000

Reference Type Variable Declarations

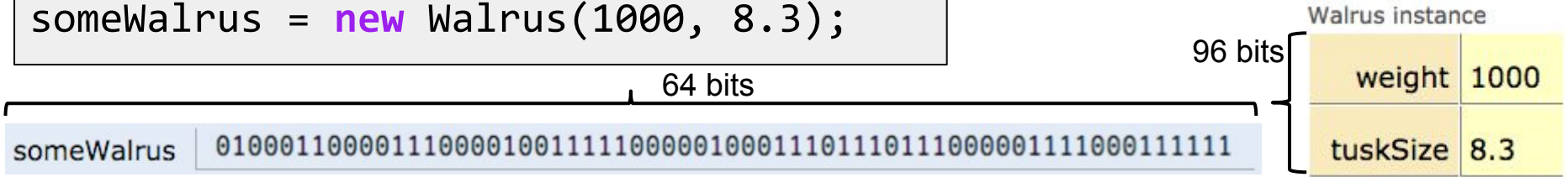
When we declare a variable of any reference type (Walrus, Dog, Planet):

- Java allocates exactly a box of size 64 bits, no matter what type of object.
- These bits can be either set to:
  - Null (all zeros).
  - The 64 bit “address” of a specific instance of that class (returned by **new**).

```
Walrus someWalrus;
someWalrus = null;
```




```
Walrus someWalrus;
someWalrus = new Walrus(1000, 8.3);
```



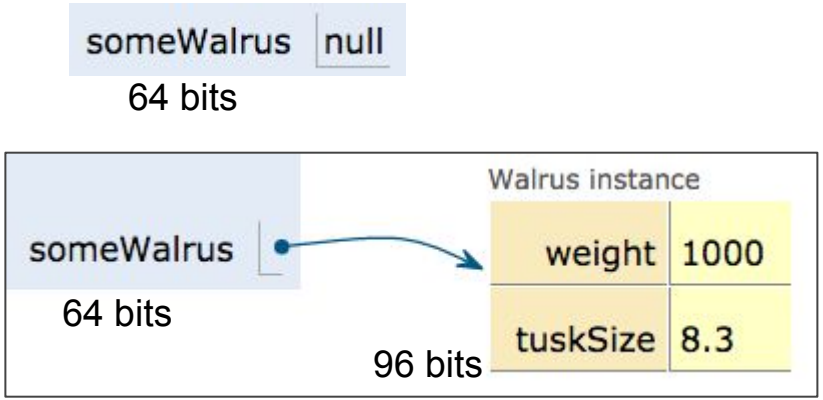
CC BY NC SA

Reference Type Variable Declarations

The 64 bit addresses are meaningless to us as humans, so we'll represent:

- All zero addresses with "null".
- Non-zero addresses as arrows.

This is sometimes called "box and pointer" notation.



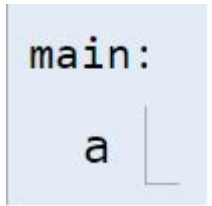
CC BY NC SA

Reference Types Obey the Golden Rule of Equals

Just as with primitive types, the equals sign copies the bits.

- In terms of our visual metaphor, we “copy” the arrow by making the arrow in the b box point at the same instance as a.

```
Walrus a;
a = new Walrus(1000, 8.3);
Walrus b;
b = a;
```



a is 64 bits

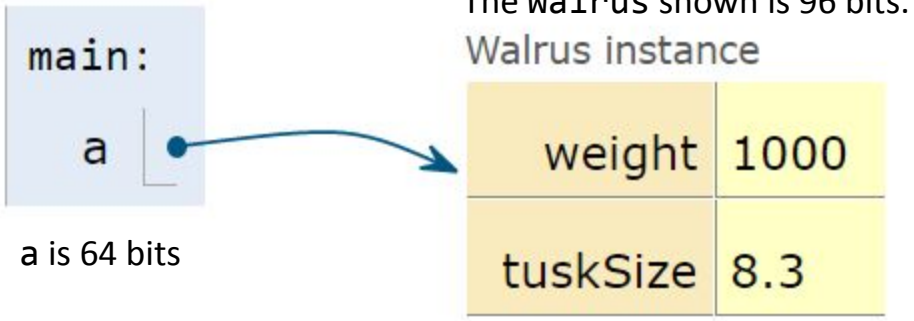
CC BY NC SA

Reference Types Obey the Golden Rule of Equals

Just as with primitive types, the equals sign copies the bits.

- In terms of our visual metaphor, we “copy” the arrow by making the arrow in the `b` box point at the same instance as `a`.

```
Walrus a;
a = new Walrus(1000, 8.3);
Walrus b;
b = a;
```



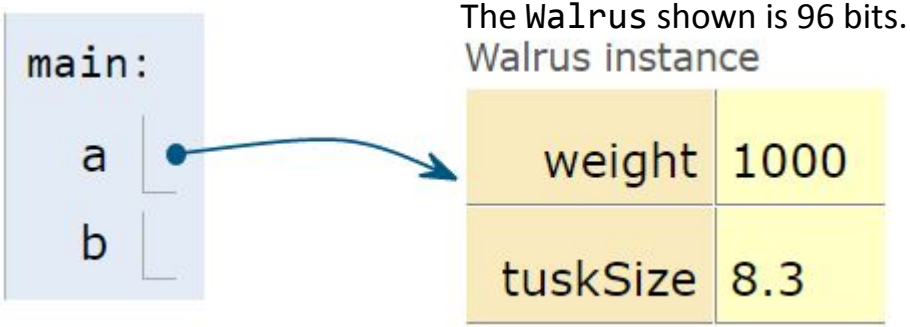
CC BY NC SA

Reference Types Obey the Golden Rule of Equals

Just as with primitive types, the equals sign copies the bits.

- In terms of our visual metaphor, we “copy” the arrow by making the arrow in the b box point at the same instance as a.

```
Walrus a;
a = new Walrus(1000, 8.3);
Walrus b;
b = a;
```



a and b are 64 bits

Note: b is currently undefined, not null!

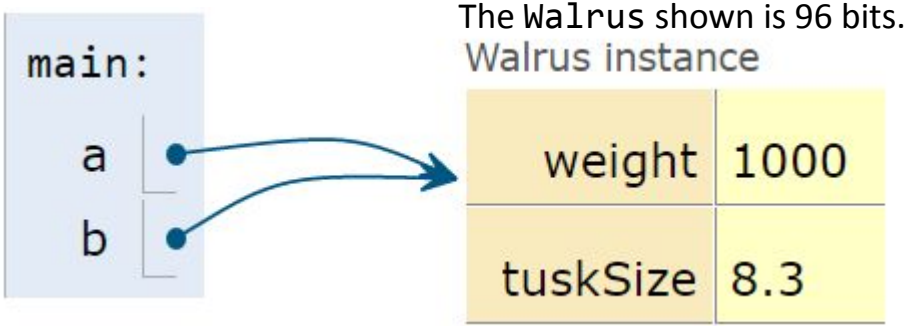
CC BY NC SA

Reference Types Obey the Golden Rule of Equals

Just as with primitive types, the equals sign copies the bits.

- In terms of our visual metaphor, we “copy” the arrow by making the arrow in the b box point at the same instance as a.

```
Walrus a;
a = new Walrus(1000, 8.3);
Walrus b;
b = a;
```



CC BY NC SA

Parameter Passing

Lecture 3, CS61B, Fall 2026

Three Important Data Structures: Lists, Arrays, and Maps

- Modern Java Lists
- Arrays
- Maps

References

- Memory Box Model
- Reference Types
- **Parameter Passing**
- 2D Arrays

CC BY NC SA

The Golden Rule of Equals (and Parameter Passing)

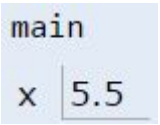
Given variables b and a:

- `b = a` **copies** all the bits from a into b.

Passing parameters obeys the same rule: Simply **copy the bits** to the new scope.

```
public static double average(double a, double b) {
   return (a + b) / 2;
}

public static void main() {
   double x = 5.5;
   double y = 10.5;
   double avg = average(x, y);
}
```



CC BY NC SA

The Golden Rule of Equals (and Parameter Passing)

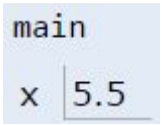
Given variables b and a:

- `b = a` **copies** all the bits from a into b.

Passing parameters obeys the same rule: Simply **copy the bits** to the new scope.

```
public static double average(double a, double b) {
   return (a + b) / 2;
}

public static void main(String[] args) {
   double x = 5.5;
   double y = 10.5;
   double avg = average(x, y);
}
```



CC BY NC SA

The Golden Rule of Equals (and Parameter Passing)

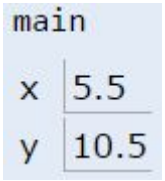
Given variables b and a:

- `b = a` **copies** all the bits from a into b.

Passing parameters obeys the same rule: Simply **copy the bits** to the new scope.

```
public static double average(double a, double b) {
   return (a + b) / 2;
}

public static void main(String[] args) {
   double x = 5.5;
   double y = 10.5;
   double avg = average(x, y);
}
```



CC BY NC SA

The Golden Rule of Equals (and Parameter Passing)

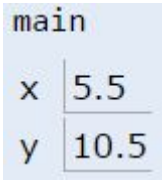
Given variables b and a:

- `b = a` **copies** all the bits from a into b.

Passing parameters obeys the same rule: Simply **copy the bits** to the new scope.

```
public static double average(double a, double b) {
    return (a + b) / 2;
}

public static void main(String[] args) {
    double x = 5.5;
    double y = 10.5;
    double avg = average(x, y);
}
```



CC BY NC SA

The Golden Rule of Equals (and Parameter Passing)

Given variables b and a:

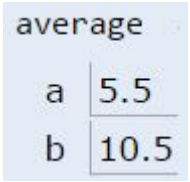
- `b = a` **copies** all the bits from a into b.

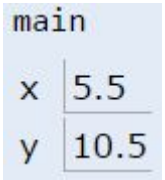
This is also called pass by value.

Passing parameters obeys the same rule: Simply **copy the bits** to the new scope.

```
public static double average(double a, double b) {
    return (a + b) / 2;
}

public static void main(String[] args) {
    double x = 5.5;
    double y = 10.5;
    double avg = average(x, y);
}
```





CC BY NC SA

The Golden Rule: Summary

There are 9 types of variables in Java:

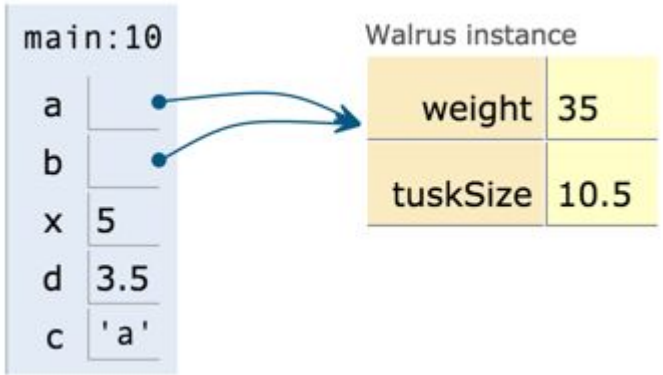
- 8 primitive types (byte, short, int, long, float, double, boolean, char).
- The 9th type is references to Objects (an arrow). References may be null.

In box-and-pointer notation, each variable is drawn as a labeled box and values are shown in the box.

- Addresses are represented by arrows to object instances.

The golden rule:

- `b = a` **copies the bits** from a into b.
- Passing parameters **copies the bits**.



CC BY NC SA

Test Your Understanding: hugcode.com/piano (NOTE THIS ONE DID NOT WORK)

Does the call to doStuff(walrus, x) have an affect on `walrus` and/or main's x?

A. Neither will change.
B. `walrus` will lose 100 lbs, but main's x will not change.
C. `walrus` will not change, but main's x will decrease by 5.
D. Both will decrease.

Answer: http://goo.gl/ngsxkq

```
public static void main(String[] args) {
   Walrus walrus = new Walrus(3500, 10.5);
   int x = 9;
   doStuff(walrus, x);
   System.out.println(walrus);
   System.out.println(x);
}
public static void doStuff(Walrus W, int x) {
   W.weight = W.weight - 100;
   x = x - 5;
}
```

CC BY NC SA

2D Arrays

Lecture 3, CS61B, Fall 2026

Three Important Data Structures: Lists, Arrays, and Maps

- Modern Java Lists
- Arrays
- Maps

References

- Memory Box Model
- Reference Types
- Parameter Passing
- **2D Arrays**

CC BY NC SA

2D Arrays: Arrays of Array Addresses

It is also possible to create two dimensional (or higher) arrays.

- Example: `int[][] x = new int[4][6];`

This creates an array called x of length 4.

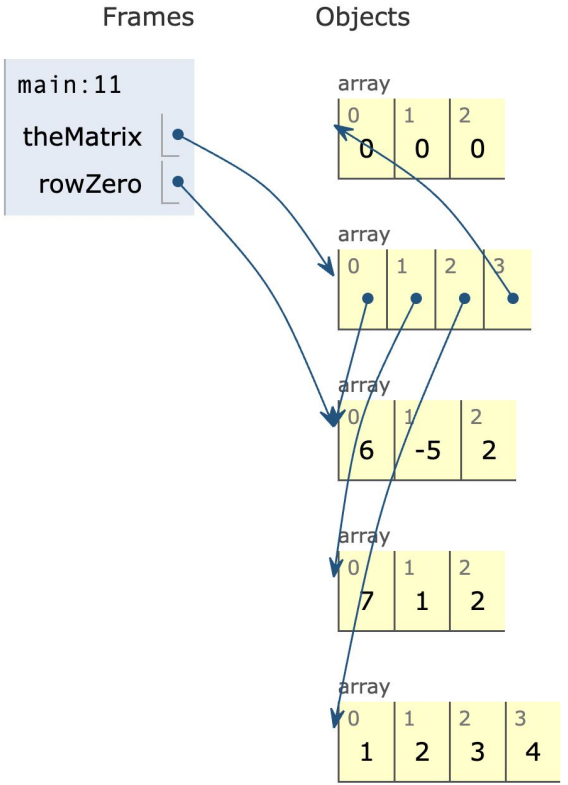
- In each entry of x, we store the address of an array of integers of length 6.

CC BY NC SA

Arrays of Array Addresses (Visualization Link)

```
int[][] theMatrix;
theMatrix = new int[4][3];
theMatrix[0] = new int[]{6, 1, 2};
theMatrix[1] = new int[]{7, 1, 2};
theMatrix[2] = new int[]{1, 2, 3, 4};

int[] rowZero = theMatrix[0];
rowZero[1] = -5;
```



CC BY NC SA

Arrays of Array Addresses (Visualization Link)

```
int[][] theMatrix;
theMatrix = new int[4][3];
theMatrix[0] = new int[]{6, 1, 2};
theMatrix[1] = new int[]{7, 1, 2};
theMatrix[2] = new int[]{1, 2, 3, 4};

int[] rowZero = theMatrix[0];
rowZero[1] = -5;
```

Array of int array references.

Create four boxes, each can store an int array reference. Then ALSO create four arrays (each of 3 memory boxes, containing 0 by default).

Create a new array with four memory boxes, storing integers 1, 2, 3, 4 respectively. Copy the address of this array into theMatrix box #2.

cc BY NC SA

One Line Version

```
int[][] pascalsTriangle = new int[][]{{1}, {1, 1},
                                      {1, 2, 1}, {1, 3, 3, 1}};
```

It is also possible to create a 2D array with the literal syntax above.

CC BY NC SA

Next Up: HW2

Make sure to finish HW2 by tomorrow.

And start project 0 once you’re done with section.

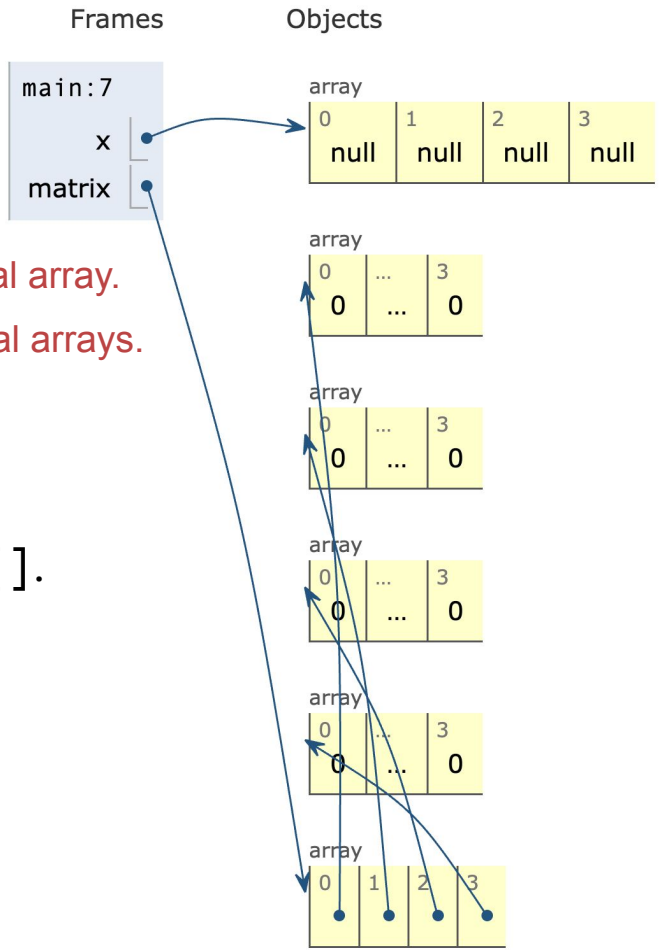
CC BY NC SA

Another Example (Visualization Link)

```
int[][] x;
int[][] matrix;
x = new int[4][];
matrix = new int[4][4];
```

Creates 1 total array.

Creates 5 total arrays.



If you want all “rows” to be the same “width”, you can specify both numbers when you instantiate the `int[][]`.

- `matrix.length` is 4.
- `matrix[i].length` is also 4 for all `i`.

CC BY NC SA

What Does This Code Do? (Bonus Slides Only Exercise)

What will be the value of x[0][0] when the code shown completes?

A. x: 1
B. x: 1
C. Other

```
int[][] x = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

int[][] z = new int[3][];
z[0] = x[0];
z[0][0] = -z[0][0];
```

Answer: https://goo.gl/CqrZ7Y

CC BY NC SA